Lesson 03 Lesson-End Project

Deploying a Kubernetes Pod Using EKS

Project agenda: To deploy Jenkins on Amazon's Elastic Kubernetes Service (EKS) using Kubernetes pod configurations

Description: Your team lead wants a CI/CD pipeline on AWS for seamless deployment of a full-stack app. The pipeline will automate the deployment of the application's frontend and backend using AWS. This ensures automatic testing and deployment of code modifications to the production environment.

Tools required: AWS Management Console

Prerequisites: You must complete lesson 06 demo 04, 03, 06, and 08

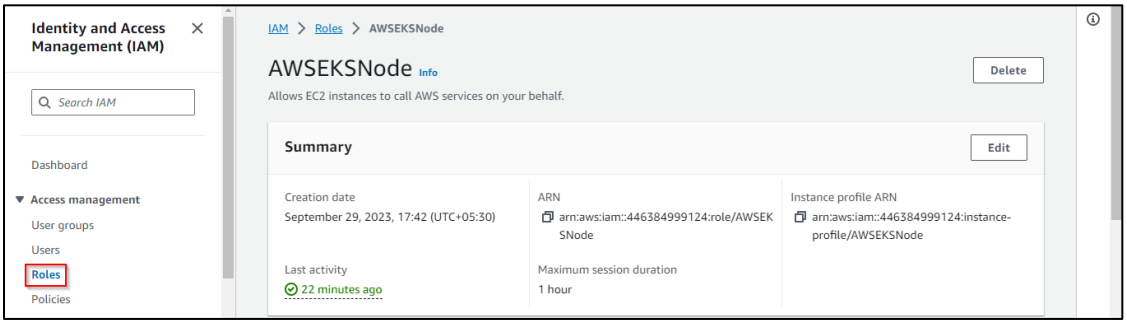
Expected deliverables: A fully operational Jenkins deployment as a Kubernetes pod on AWS EKS, accessible externally

Steps to be followed:

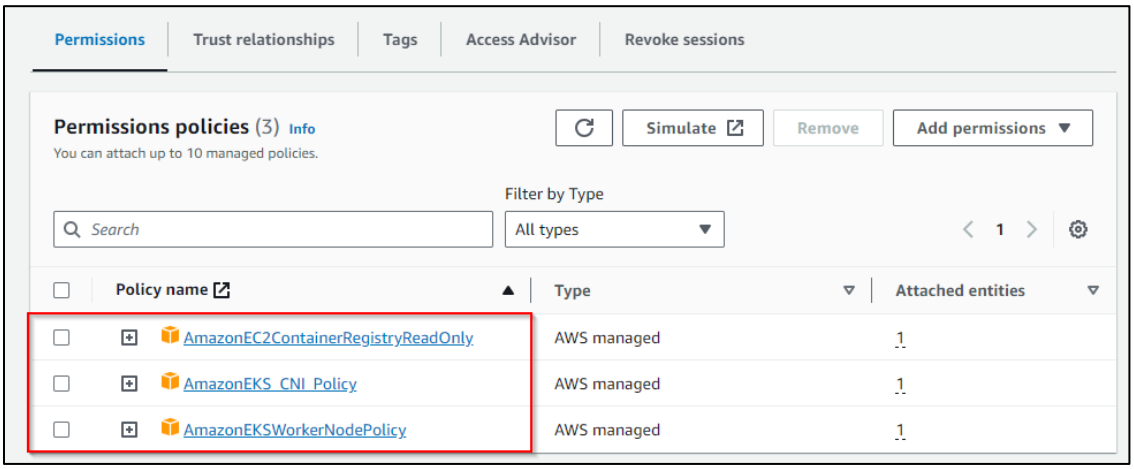
1. Deploy Jenkins on EKS as a Kubernetes Pod

Step 1: Deploy Jenkins on EKS as a Kubernetes Pod

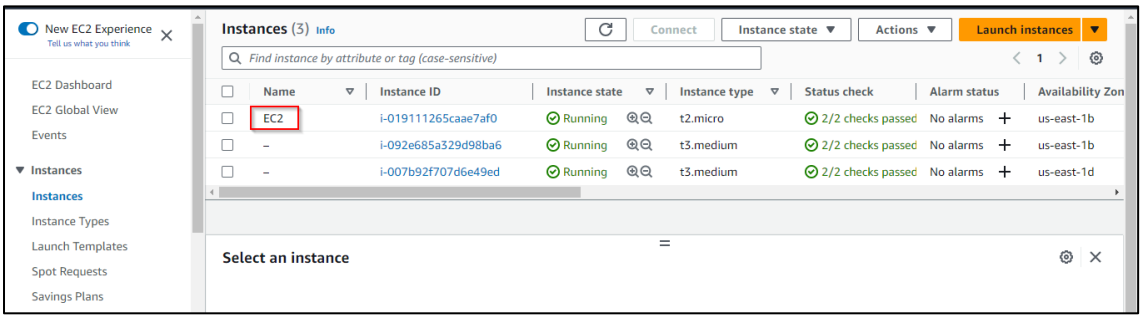
1.1 In the AWS console, navigate to the **IAM** section and click on **Roles**



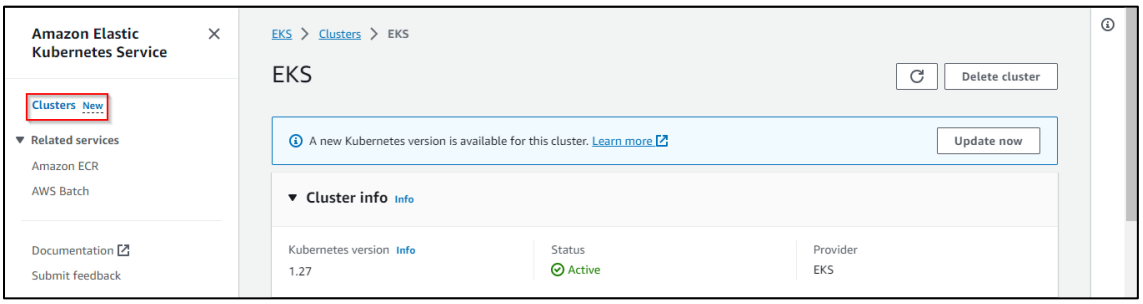
1.2 Create a new IAM role and add the following policies shown in the screenshot below:

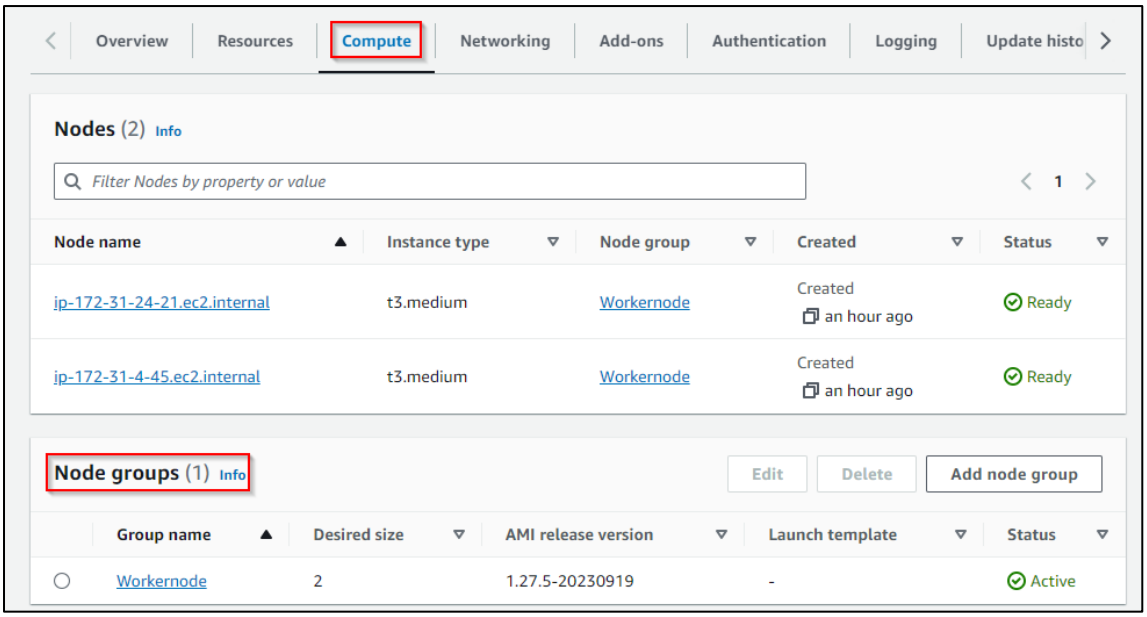


1.3 Navigate to the EC2 dashboard to initiate the creation of a new EC2 instance



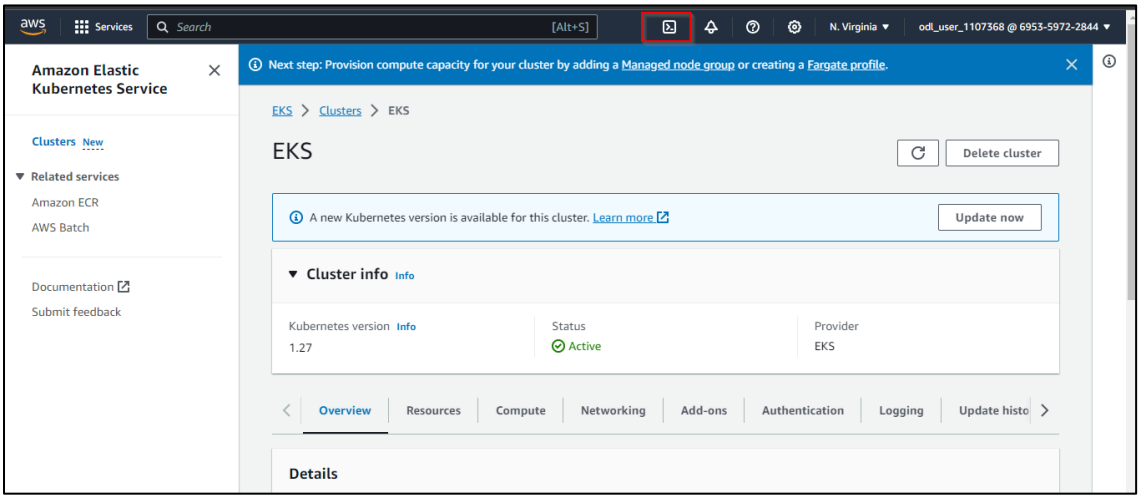
1.4 Navigate to the EKS dashboard, initiate the creation of a new Cluster, and click on **Node groups** to create new nodes





Note: Refer to the previous demos for a comprehensive guide on creating IAM roles, EC2 instances, and EKS.

1.5 Launch the AWS **CloudShell** by clicking on its icon



1.6 Input the following commands to update the EKS Clusters:

kubectl get node

aws eks update-kubeconfig --name [YOUR_CLUSTER_NAME]

```
[cloudshell-user@ip-10-2-65-241 ~]$ aws eks update-kubeconfig --name EKS
Updated context arn:aws:eks:us-east-1:446384999124:cluster/EKS in /home/cloudshell-user/.kube/config
[cloudshell-user@ip-10-2-65-241 ~]$ kubectl get node
NAME                            STATUS   ROLES    AGE     VERSION
ip-172-31-24-21.ec2.internal    Ready    <none>   7m17s   v1.27.5-eks-43840fb
ip-172-31-4-45.ec2.internal     Ready    <none>   7m25s   v1.27.5-eks-43840fb
```

```
[cloudshell-user@ip-10-6-100-80 ~]$ aws eks update-kubeconfig --name EKS
Added new context arn:aws:eks:us-east-1:695359722844:cluster/EKS to /home/cloudshell-user/.kube/config
[cloudshell-user@ip-10-6-100-80 ~]$
```

Note: Ensure you replace **[YOUR_CLUSTER_NAME]** with the name of the cluster you created.

1.7 Create a new **deployment.yaml** file for your Jenkins deployment and enter the following code:

nano deployment.yaml

```
[cloudshell-user@ip-10-2-65-241 ~]$ nano deployment.yaml
```

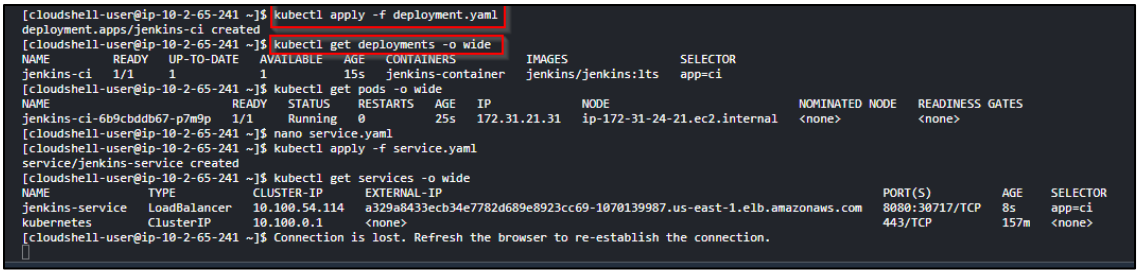
```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: jenkins-ci
  labels:
    product: ci
spec:
  replicas: 1
  selector:
    matchLabels:
      app: ci
  template:
    metadata:
      labels:
        app: ci
```

```
spec:

  containers:
  - name: jenkins-container
    image: jenkins/jenkins:lts
    ports:
    - containerPort: 8080
    resources:
      limits:
        cpu: 1000m
        memory: 600Mi
      requests:
        cpu: 500m

        memory: 300Mi
```

1.8 Run the following commands to apply the deployment configuration from deployment.yaml and then list all deployments with detailed information

kubectl apply -f deployment.yaml

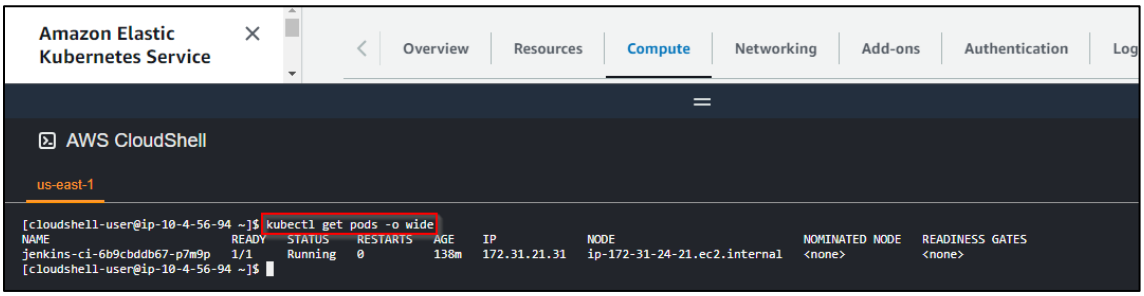
kubectl get deployments -o wide



```
[cloudshell-user@ip-10-2-65-241 ~]$ kubectl apply -f deployment.yaml
deployment.apps/jenkins-ci created
[cloudshell-user@ip-10-2-65-241 ~]$ kubectl get deployments -o wide
NAME         READY   UP-TO-DATE   AVAILABLE   AGE   CONTAINERS          IMAGES                SELECTOR
jenkins-ci   1/1     1            1           15s   jenkins-container   jenkins/jenkins:lts   app=ci
[cloudshell-user@ip-10-2-65-241 ~]$ kubectl get pods -o wide
NAME                          READY   STATUS    RESTARTS   AGE   IP             NODE                            NOMINATED NODE   READINESS GATES
jenkins-ci-6b9cbddb67-p7m9p   1/1     Running   0          25s   172.31.21.31   ip-172-31-24-21.ec2.internal   <none>           <none>
[cloudshell-user@ip-10-2-65-241 ~]$ nano service.yaml
[cloudshell-user@ip-10-2-65-241 ~]$ kubectl apply -f service.yaml
service/jenkins-service created
[cloudshell-user@ip-10-2-65-241 ~]$ kubectl get services -o wide
NAME              TYPE           CLUSTER-IP      EXTERNAL-IP                                                              PORT(S)          AGE   SELECTOR
jenkins-service   LoadBalancer   10.100.54.114   a329a8433ecb34e7782d689e8923cc69-1070139987.us-east-1.elb.amazonaws.com   8080:30717/TCP   8s    app=ci
kubernetes        ClusterIP      10.100.0.1      <none>                                                                   443/TCP          157m  <none>
[cloudshell-user@ip-10-2-65-241 ~]$ Connection is lost. Refresh the browser to re-establish the connection.
```

1.9 To view the pods, use the following command:

kubectl get pods -o wide



After successfully deploying Jenkins, you can create a service with a Load Balancer to access Jenkins externally.

1.10 Create another **.yaml** file for the Jenkins service and paste the following code into the editor:

nano service.yaml

```
[cloudshell-user@ip-10-134-41-224 ~]$ nano service.yml
```

```
apiVersion: v1
kind: Service
metadata:
  name: jenkins-service
  labels:
    app: jenkins
spec:
  ports:
   - name: jenkins-container
     port: 8080
     targetPort: 8080
  type: LoadBalancer
  selector:
    app: ci
```

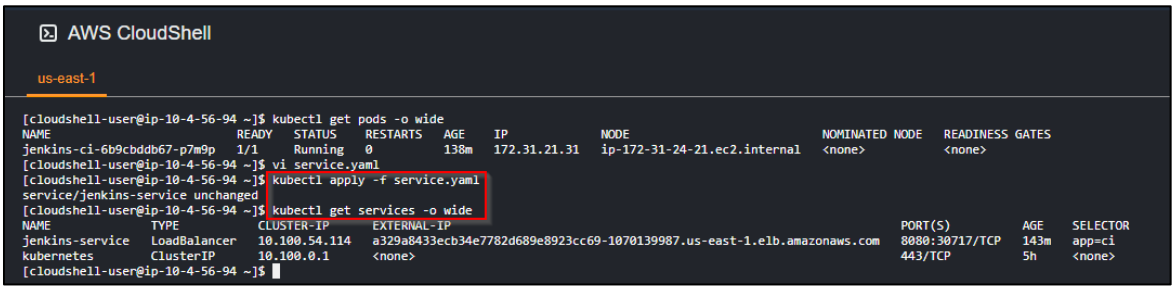
```
apiVersion: v1
kind: Service
metadata:
  name: jenkins-service
  labels:
    app: jenkins
spec:
  ports:
   - name: jenkins-container
     port: 8080
     targetPort: 8080
  type: LoadBalancer
  selector:
    app: ci
~
~
~
```

To save and exit the editor, press **shift** + **:** , type wq, and press **Enter**

1.11 Deploy the service and view its details using the below commands:

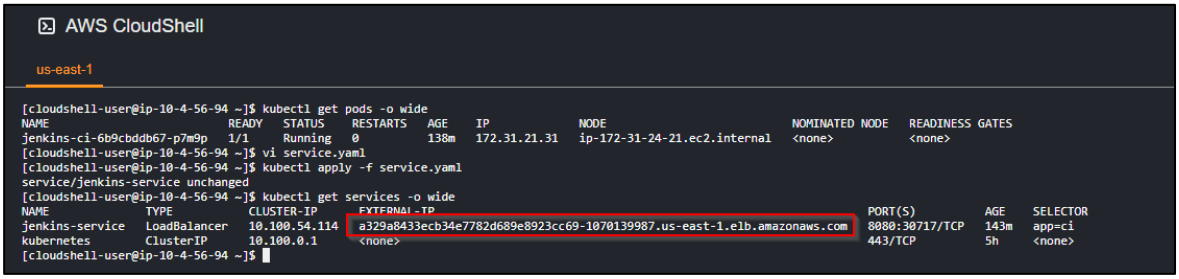
kubectl apply -f service.yaml

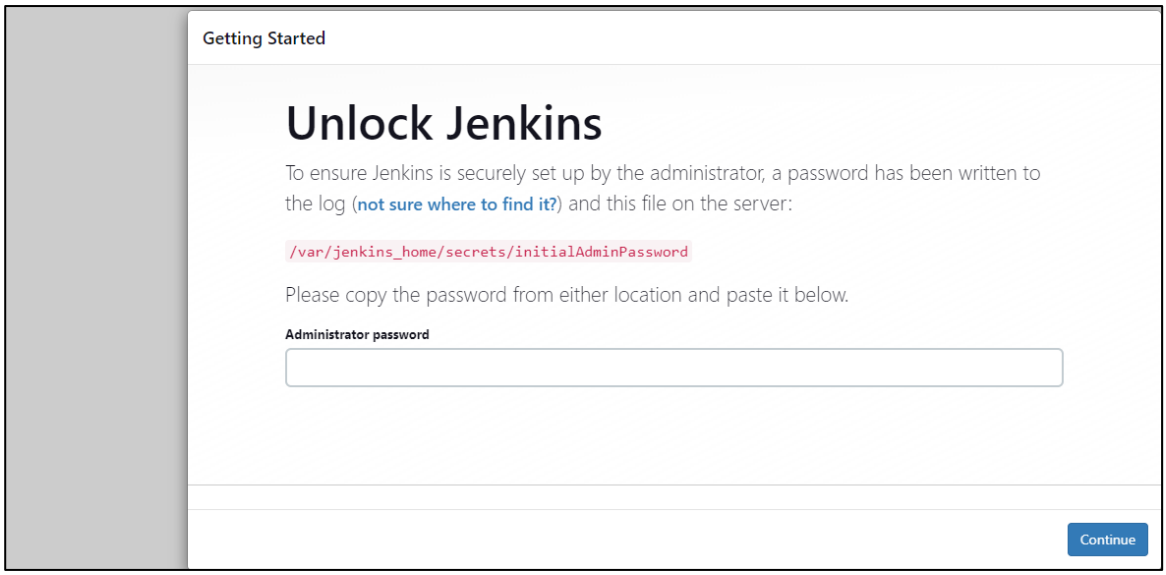
kubectl get services -o wide



Given that a Load Balancer service type is utilized, you can retrieve the External IP details, which correspond to an ELB Load Balancer, to access the Jenkins application.

1.12 To access Jenkins, utilize the following Load Balancer URL combined with port **8080** in your browser: **http://a329a8433ecb34e7782d689e8923cc69-1070139987.us-east-1.elb.amazonaws.com:8080**





By following these steps, you have successfully deployed Jenkins as a Kubernetes pod on Amazon EKS and established an externally accessible Jenkins instance. This enables you to efficiently manage and automate your projects using this powerful continuous integration tool.